República Bolivariana de Venezuela
Universidad Central de Venezuela
Facultad de Ciencias
Escuela de Computación
Organización y Estructura del Computador

INFORME:
PRIMERA ETAPA DEL PROYECTO (RISC-16)

Profesores

Antonio Machado
Gustavo Torres
Aparicio Peña

Estudiantes

José Maceiras
C.I. 31.228.863

Santiago Flores
C.I. 32.606.881

Daniel Reyes
C.I. 32.303.416

Caracas, 23 de enero de 2026

La arquitectura de este ensamblador comienza con la definición del espacio de memoria. La dirección inicial se determina partiendo del requerimiento establecido en el enunciado, basado en la suma de las cédulas de identidad de los integrantes del equipo. Tras sumar los valores 31.228.863, 32.303.416 y 32.606.881, se obtuvo un total de 96139160. Debido a que el procesador opera con un ancho de palabra de 16 bits, este valor se truncó aplicando un módulo 2^16 (96139160 mod 65536), resultando en la dirección decimal 63384, o su equivalente hexadecimal 0xF798. Este valor actúa como el punto de origen para el Contador de Programa (PC) y como el punto de anclaje para todas las etiquetas relativas generadas durante el ensamblaje.

Para abordar la complejidad de un simulador de arquitectura, se optó por un enfoque robusto basado en la Programación Orientada a Objetos en C++. Dado que las especificaciones del proyecto no imponían restricciones estrictas sobre el uso de bibliotecas estándar, decidimos aprovechar la potencia de la Standard Template Library (STL). Tras un proceso de investigación sobre la implementación de las estructuras de datos, decidimos usar std::map para simular tanto la memoria RAM como la tabla de símbolos. Asimismo, se utilizó std::vector para almacenar la secuencia de instrucciones, permitiendo un acceso rápido durante la simulación.

La entrada del programa, al ver que no tendría sentido procesar instrucciones por entrada básica al estar trabajando con la lectura de un lenguaje ensamblador, es estrictamente por archivo, pero el usuario puede proporcionar la ruta tanto por parámetros de línea de comandos como de forma interactiva tras la ejecución. Ejemplos:

```
$ ./grupo-assemblyboys-exe-f1 test.asm
```

```
$ ./grupo-assemblyboys-exe-f1
Ingrese el nombre del archivo (.asm): test.asm
```

Para interpretar el código fuente en ensamblador, diseñamos un tokenizador personalizado que actúa como la primera línea de procesamiento. La función que se implementó se enfoca en dividir las cadenas de texto y también en segmentar cada línea del archivo .asm eliminando caracteres prescindibles como comas y espacios en blanco, separando los nemónicos de sus operandos y almacenarlos en un vector. Una característica de nuestra implementación es la flexibilidad en la lectura de inmediatos: el programa es capaz de reconocer valores numéricos tanto si se presentan como números puros o si llevan el prefijo $ (excepto en las traducciones de nemónicos que no existen en hardware), gracias a una función de limpieza de strings que limpia la entrada antes de convertirla a valores enteros. También se soportan comentarios con punto y coma (;).

Utilizamos polimorfismo para representar las instrucciones. Definimos una clase base Instruccion de la cual heredan InstruccionR, InstruccionI y InstruccionJ. Este enfoque nos permite que cada tipo de instrucción maneje su propia lógica de ensamblaje (manipulación de bits para generar el código máquina) y de ejecución (alteración del estado de la CPU). Por ejemplo, las instrucciones de Tipo R se encargan de los desplazamientos de bits para los registros Rs1, Rs2 y Rd, mientras que las de Tipo J gestionan el cálculo del offset relativo necesario para los saltos.

Tras un proceso de investigación nos decantamos por hacer un ensamblador de dos pasadas. En la primera pasada, el sistema recorre el archivo .asm limpiando comentarios y espacios, identificando directivas como .org o .word y, lo más importante, construyendo la tabla de símbolos que asocia cada etiqueta con una dirección de memoria específica.

ORGA_PROYECTO - mic.cc

```cpp
// ------------------------------------------------------------
//              PRIMERA PASADA DEL ENSAMBLADOR
// ------------------------------------------------------------

void primeraPasada(const string& archivo) {
    ifstream in(archivo);
    if (!in) {
        cout << "No se pudo abrir el archivo\n";
        exit(1);
    }

    string linea;
    uint16_t pc = DIR_INICIAL;

    while (getline(in, linea)) {
        linea = limpiarLinea(linea);
        if (linea.empty()) continue;

        // Etiqueta
        if (linea.back() == ':') {
            tablaSimbolos[linea.substr(0, linea.size() - 1)] = pc; // dfdadf
            continue;
        }

        vector<string> tokens = tokenizar(linea);
        if (tokens.empty()) continue;

        // Directiva
        if (tokens[0][0] == '.') {
            procesarDirectiva(tokens, pc, linea);
            continue;
        }

        // Instruccion
        procesarInstruccion(tokens, pc, linea);
        pc++;
    }
}
```

En la segunda pasada, se realiza la traducción real: cada nemónico se convierte en su código de operación (COP) correspondiente y dependiendo del tipo de las instrucción, mediante manipulación de bits y cálculos de offsets, y con los datos que hemos recolectado y almacenado en la primera pasada (y que también hemos guardado en nuestros objetos de cada instrucción correspondiente gracias al tokenizador) ya podemos generar el código de máquina correspondiente a la instrucción de ensamblador.

ORGA_PROYECTO - mic.cc

```cpp
// ------------------------------------------------------------
//              SEGUNDA PASADA DEL ENSAMBLADOR
//             Generacion del codigo de maquina
// ------------------------------------------------------------

void segundaPasada() {
    // itera sobre todas las instrucciones del programa y llama a su ensamblar ensamblar() en cada una
    for (Instruccion *&inst : programa) {
        inst->ensamblar();
    }
}
```

También durante la segunda pasada la Tabla de Símbolos ya está completa (gracias a la primera pasada). Esto permite resolver saltos hacia adelante, es decir, saltar a una etiqueta que estaba definida más abajo en el código. Gracias al polimorfismo, el bucle principal de la segunda pasada es extremadamente limpio.

Esta es la etapa crítica del ensamblador. Una vez que la tabla de símbolos está completa y todas las direcciones de memoria son conocidas, el sistema itera nuevamente sobre la lista de instrucciones para generar los 16 bits correspondientes al código máquina de cada operación.

La implementación de esta fase se apoya en el método virtual ensamblar(), el cual encapsula la lógica específica de codificación para cada formato.

La construcción del código binario se realiza mediante operaciones de desplazamiento (<<) y compuertas lógicas OR (|). Esto asegura que cada campo de la instrucción (Código de Operación, Registros, Inmediatos) se ubique en los bits exactos que dicta la arquitectura RISC-16.

Por ejemplo, para una instrucción Tipo R, el código se ensambla fusionando los campos:

```
void ensamblar() override {
    // Construcción del código máquina mediante desplazamientos
    codigo = 0;
    codigo |= (cop & 0xF) << 12; // Bits 15-12: COP
    codigo |= (0b00) << 10;      // Bits 11-10: Tipo 00
    codigo |= (rs1 & 0x7) << 7;  // Bits 9-7: Rs1
    codigo |= (rs2 & 0x7) << 4;  // Bits 6-4: Rs2
    codigo |= (rd & 0x7);        // Bits 3-0: Rd
}
```

El desafío técnico más importante de la segunda pasada es la resolución de direcciones relativas para instrucciones de control de flujo (BEQ, JMP, BNE).

La arquitectura RISC-16 que estamos desarrollando utiliza direccionamiento relativo al Program Counter (PC). En la primera pasada, solo se conocía el nombre de la etiqueta destino. En la segunda pasada el método InstruccionJ::ensamblar() realiza el cálculo aritmético para determinar el desplazamiento necesario:

1. Se recupera la dirección absoluta de la etiqueta destino desde la tablaSimbolos.
2. Se calcula la diferencia con respecto a la dirección de la instrucción actual:

   offset = (int8_t)(target - direccion);

3. El resultado se trunca a 8 bits (o 7 bits según el formato) y se empaqueta en la instrucción.

Este mecanismo permite que el ensamblador soporte saltos tanto hacia atrás (los bucles) como hacia adelante (los condicionales) de manera transparente para el programador.

De manera similar, las instrucciones de acceso a memoria (LW, SW) pueden utilizar etiquetas para referenciar variables en la sección de datos. Durante la segunda pasada, si el operando es una etiqueta en lugar de un número, el ensamblador calcula la distancia relativa en memoria para acceder al dato correcto, convirtiendo la abstracción del nombre de la variable en un desplazamiento numérico inmediato.

Dentro de nuestra lógica de ejecución, existe una dependencia entre las instrucciones JAL y LW. En la arquitectura RISC-16, JAL tiene la función crítica de guardar la dirección de retorno (PC + 1) en el registro R7 antes de realizar un salto. En nuestra implementación, la presencia de un JAL antes de un LW es bastante necesaria ya que no tenemos acceso al PC directamente, lo que nos dificulta el cálculo que debe realizarse para poder leer una dirección de memoria y poder guardar así los datos en el registro. Pasa algo similar con SW.

```
ORIGEN:
.word 1234
```

```
    JAL BASE_PC
BASE_PC:
    LW R1, ORIGEN(R7)
```

Tras haber definido todas las direcciones de memoria y los métodos de cada instrucción, venimos a la siguiente fase que sería de simulación. Se realizó un intérprete dinámico que procesa el código máquina generado previamente. Este proceso inicia con una carga preliminar de datos en la cual se recorren todas las estructuras de la memoria para colocar los valores definidos mediante directivas, como palabras o cadenas de caracteres, en sus direcciones correspondientes dentro de un mapa que emula la memoria RAM. Una vez preparada la memoria, el sistema entra en un ciclo de ejecución continuo que emula el comportamiento de un procesador real, basándose estrictamente en el valor del registro contador de programa o PC. En cada iteración de este ciclo, la CPU realiza una búsqueda dentro del conjunto de instrucciones para localizar aquella cuya dirección coincida con el PC actual, lo que equivale a la etapa de búsqueda o fetch de un procesador físico. Si no se encuentra una instrucción válida en dicha dirección, el simulador detiene su ejecución para prevenir comportamientos erráticos, garantizando la integridad de la emulación.

```cpp
void simular(vector<Instruccion*>& programa, CPU &cpu, Memoria &mem) {
    // iteramos sobre todas las instrucciones para cargar los datos de las directivas en la memoria virtual
    for (Instruccion* inst : programa) {
        inst->cargarDatos(mem);
    }

    while (!cpu.halted) {
        Instruccion* inst = nullptr;

        // busqueda de la instruccion en la lista segun el PC actual
        for (Instruccion* i : programa) {
            if (i->getDireccion() == cpu.PC) {
                inst = i;
                break;
            }
        }

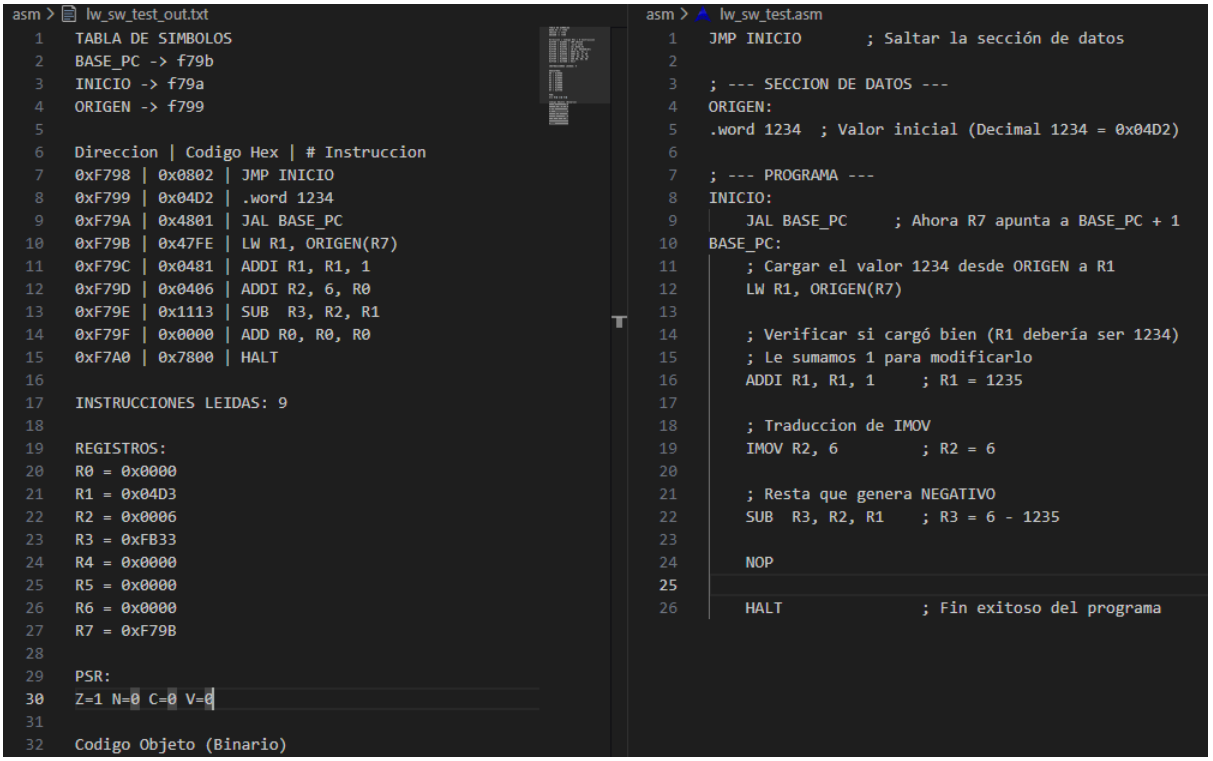
        if (!inst) {
            cout << "PC invalido: " << hex << cpu.PC << endl;
            break;
        }

        inst->ejecutar(cpu, mem); // cada instruccion llama a su funcion ejecutar() correspondiente segun el tipo
    }
}
```

Cuando una instrucción es localizada, se activa el mecanismo de ejecución polimórfico, donde cada objeto de instrucción realiza las operaciones lógicas, aritméticas o de control de flujo pertinentes según su tipo. En este punto, la simulación interactúa profundamente con el banco de registros y la unidad lógica, calculando resultados y, lo que es más crítico, actualizando el registro de estado o PSR. Esta actualización implica una lógica matemática compleja para determinar condiciones de cero, signo negativo, acarreo y desbordamiento, permitiendo que el software virtual tome decisiones basadas en los resultados de operaciones previas. En el caso de las instrucciones de salto o control de flujo, el simulador altera el orden secuencial del contador de programa, permitiendo la ejecución de bucles y

estructuras condicionales que definen la lógica del programa ensamblado. El proceso prosigue de forma cíclica hasta que se encuentra una instrucción de detención específica, denominada HALT, la cual cambia el estado interno de la CPU para finalizar la emulación y proceder a la generación de los reportes de salida con el estado final del sistema.

Finalmente, después de haberse simulado la ejecución, el sistema genera un archivo de salida .txt que cumple con el formato de tres columnas solicitado: Dirección | Código Hexadecimal | Instrucción original. El informe final del archivo muestra el estado de todos los registros y las banderas del PSR, proporcionando una visión completa del estado del programa generado, además del código máquina en binario al final.



asm > lw_sw_test_out.txt

```
TABLA DE SIMBOLOS
BASE_PC -> f79b
INICIO -> f79a
ORIGEN -> f799

Direccion | Codigo Hex | # Instruccion
0xF798 | 0x0802 | JMP INICIO
0xF799 | 0x04D2 | .word 1234
0xF79A | 0x4801 | JAL BASE_PC
0xF79B | 0x47FE | LW R1, ORIGEN(R7)
0xF79C | 0x0481 | ADDI R1, R1, 1
0xF79D | 0x0406 | ADDI R2, 6, R0
0xF79E | 0x1113 | SUB  R3, R2, R1
0xF79F | 0x0000 | ADD R0, R0, R0
0xF7A0 | 0x7800 | HALT

INSTRUCCIONES LEIDAS: 9

REGISTROS:
R0 = 0x0000
R1 = 0x04D3
R2 = 0x0006
R3 = 0xFB33
R4 = 0x0000
R5 = 0x0000
R6 = 0x0000
R7 = 0xF79B

PSR:
Z=1 N=0 C=0 V=0

Codigo Objeto (Binario)
```

asm > lw_sw_test.asm

```
JMP INICIO       ; Saltar la sección de datos

; --- SECCION DE DATOS ---
ORIGEN:
.word 1234  ; Valor inicial (Decimal 1234 = 0x04D2)

; --- PROGRAMA ---
INICIO:
    JAL BASE_PC     ; Ahora R7 apunta a BASE_PC + 1
BASE_PC:
    ; Cargar el valor 1234 desde ORIGEN a R1
    LW R1, ORIGEN(R7)

    ; Verificar si cargó bien (R1 debería ser 1234)
    ; Le sumamos 1 para modificarlo
    ADDI R1, R1, 1     ; R1 = 1235

    ; Traduccion de IMOV
    IMOV R2, 6         ; R2 = 6

    ; Resta que genera NEGATIVO
    SUB  R3, R2, R1    ; R3 = 6 - 1235

    NOP

    HALT               ; Fin exitoso del programa
```